Relatório Maratona ENADE

Documentação técnica do projeto em R – funcionamento, scripts e histórico

Repositório: abrittaa/Relatorio_Maratona_ENADE · Gerado em 28/10/2026

Visão Geral

O projeto consolida os resultados dos testes da **Maratona ENADE** respondidos por alunos de várias universidades (arquivos CSV exportados da plataforma de testes). A partir deles, gera planilhas consolidadas e um **portal HTML** em que cada aluno consulta o próprio desempenho e emite um certificado.

1. Estrutura do Projeto

Arquivo / Pasta	Função
`criar_dados.R`	Lê todos os CSVs e cria a base bruta `dados_completos`
`agrupar_dados.R`	Consolida **1 linha por aluno** (`dados_limpos`)
`agrupar_por_teste.R`	Consolida **1 linha por aluno × teste** (`dados_per_teste`)
`verificacao_de_realizacao_do_teste.R`	Verifica quem fez / não fez cada teste
`geracao_relatorio.R`	Orquestra tudo e gera o `portal_alunos.html`
`Dados_testes/Testes respondidos/Teste XX/`	CSVs no formato `Teste XX <Universidade>.csv` (criptografados com **git-crypt**)
`Dados_testes/Universidades participantes da Maratona/`	Planilhas das universidades (ainda não utilizadas pelos scripts)

Pacotes utilizados: `tidyverse` , `tools` , `magrittr` , `writexl` , `jsonlite` , `digest` .

2. Fluxo de Execução (Pipeline)



Rodar `geracao_relatorio.R` executa todo o pipeline, pois ele faz `source()` dos quatro scripts na ordem acima.

3. Detalhamento de Cada Script

3.1 criar_dados.R

1. Lista recursivamente todos os `.csv` em `Dados_testes/Testes respondidos`.
2. Para cada arquivo:
   - Lê todas as colunas como texto (`col_types = "c"`);
   - Cria `id_teste` = nome da pasta (ex.: `"Teste 01"`);
   - Cria `faculdade` = nome do arquivo sem o prefixo `Teste XX` (regex `^(?i)teste[ _]*\d+[ _]*`), ex.: `"UNIFEI Itajuba"`.
3. Empilha tudo em **`dados_completos`** (`map_dfr`).
4. Cria também um objeto separado por teste no ambiente global (`teste_01`, `teste_02`, ...) via `group_split` + `assign`.

3.2 agrupar_dados.R — 1 linha por aluno

- Agrupa por `Name` + `faculdade`.
- **Soma** `Points Earned` e `Points Possible` de todos os testes e recalcula `Percent = obtidos / possíveis × 100` (1 casa decimal).
- Nas demais colunas aplica `obter_valor_valido()`: retorna o primeiro valor que não seja NA, `"NULL"`, `"NA"` ou vazio; caso não exista, retorna `"Não Realizou o Teste"`.
- `formatar_cabecalho()` encurta os nomes das colunas de questões para 4 caracteres sem espaços (ex.: `"GP 01..."` → `GP01`) e desambigua duplicatas com `_1`, `_2` (`make.unique`).
- Exporta `dados_limpos.csv` e `dados_limpos.xlsx`.

3.3 agrupar_por_teste.R — 1 linha por aluno × teste

- Agrupa por `faculdade`, `Name`, `id_teste`; soma pontos, calcula o `Percent` do teste e guarda `Time`, `Start Datetime` e `End Datetime`.
- Cria `dados_per_teste_matriz` (`pivot_wider`): uma linha por aluno e uma coluna `Nota_%_Teste XX` para cada teste.
- Exporta CSV e XLSX das duas tabelas.

3.4 verificacao_de_realizacao_do_teste.R

- Se `dados_completos` não existir, executa `criar_dados.R` (único script com essa proteção).
- `relatorio_testes_alunos`: por aluno, a quantidade de testes realizados, o percentual sobre o total e a lista de testes **não respondidos** (ou `"Nenhum (Respondeu a todos)"`).
- `matriz_status_testes`: tabela aluno × teste com "*Respondido*" / "*Não Respondido*".
- Exporta CSV e XLSX.

3.5 geracao_relatorio.R — Portal do Aluno

1. Executa os quatro scripts anteriores.
2. **Hash LGPD:** normaliza o nome (trim, minúsculas, espaços únicos) e gera **SHA-256** com `digest`.
3. Monta para cada aluno um registro com: pontos, % geral, testes concluídos/total, progresso, testes pendentes, detalhes por teste (Concluído/Pendente), código do certificado (`CERT-` + 10 primeiros caracteres do hash) e data de emissão.
4. Converte os registros para JSON (`jsonlite::toJSON`) e embute num HTML único com CSS e JavaScript próprios.
5. **No navegador:** o aluno digita o nome completo → o JavaScript calcula o SHA-256 do nome normalizado (`crypto.subtle`) → se o hash existir, exibe o painel (métricas, barra de progresso, alerta de pendências, tabela por teste) e o certificado personalizado. O botão *Imprimir / Salvar PDF* imprime apenas o certificado (`@media print`).
6. Salva o arquivo `portal_alunos.html`.

4. Arquivos Gerados

Script	Saídas
agrupar_dados.R	`dados_limpos.csv`, `dados_limpos.xlsx`
agrupar_por_teste.R	`dados_per_teste.csv/.xlsx`, `dados_per_teste_matriz.csv/.xlsx`
verificacao_de_realizacao_do_teste.R	`relatorio_testes_alunos.csv/.xlsx`, `matriz_status_testes_matriz.csv/.xlsx`
geracao_relatorio.R	`portal_alunos.html`

> Os arquivos `*.csv`, `*.xlsx`, `*.html` e `*.key` são ignorados pelo Git (`.gitignore`). A pasta `Dados_testes/**` é criptografada via **git-crypt** (`.gitattributes`).

5. Histórico de Operações (Git)

Data	Operação
14/10	Criação do projeto (`.Rproj` , `.gitignore`) e do script `criar_dados.R`
14/10	Primeira versão de `agrupar_dados.R` ; merge com o repositório remoto; correção do bug de valores `"NULL"`
14/10	Proteção de dados com git-crypt e inclusão dos CSVs/planilhas das universidades
15/10	Correção do cálculo da porcentagem de acerto (soma de pontos); remoção de coluna desnecessária
15/10	Criação de `agrupar_por_teste.R` ; `.csv` / `.xlsx` adicionados ao `.gitignore`
25/10	`*.key` ignorado (chave de criptografia); criação de `verificacao_de_realizacao_do_teste.R`
25/10	Criação de `geracao_relatorio.R` ; o HTML gerado foi commitado e depois revertido; `*.html` passa a ser ignorado

6. Pontos de Atenção Identificados

> **LGPD / Privacidade:** o hash SHA-256 é usado apenas como *chave* do dicionário, mas cada registro ainda contém `nome` , `faculdade` e notas em texto puro dentro do HTML. Qualquer pessoa que abrir o código-fonte da página consegue ver os dados de todos os alunos. Para proteção real, seria necessário criptografar cada registro (ex.: AES com chave derivada do nome) ou remover nome e notas do JSON.

1. **Faculdades duplicadas:** arquivos como `Teste 06 Mackenzie (Sem resposta).csv` geram `faculdade = "Mackenzie (Sem resposta)"` (e variação com "sem" minúsculo), separando alunos do Mackenzie em grupos diferentes.
2. **Pasta `teste 05` em minúsculo:** gera `id_teste = "teste 05"` , inconsistente com os demais, o que pode afetar a ordenação e a exibição no portal.
3. **Homônimos:** o hash usa apenas o nome; dois alunos com o mesmo nome em faculdades diferentes se sobrescrevem no portal.
4. **Dependências implícitas:** `agrupar_dados.R` e `agrupar_por_teste.R` falham se executados isoladamente (dependem de `dados_completos`). Além disso, `geracao_relatorio.R` reexecuta `criar_dados.R` sempre e regrava todos os CSV/XLSX.
5. **Código do certificado:** é determinístico e derivado do nome, portanto qualquer pessoa pode recalculá-lo; não comprova autenticidade.